

A practical guide to designing, communicating, and defending your system architecture — for undergraduate capstone students and faculty advisors.

[illegible]

What We'll Cover

01

Capstone Context

Where system architecture fits in your project and research deliverables

02

What Is a System Architecture Diagram?

Definition, purpose, and what questions it answers

03

Common Types & Styles

Context, layered, client-server, microservices, and deployment diagrams

04

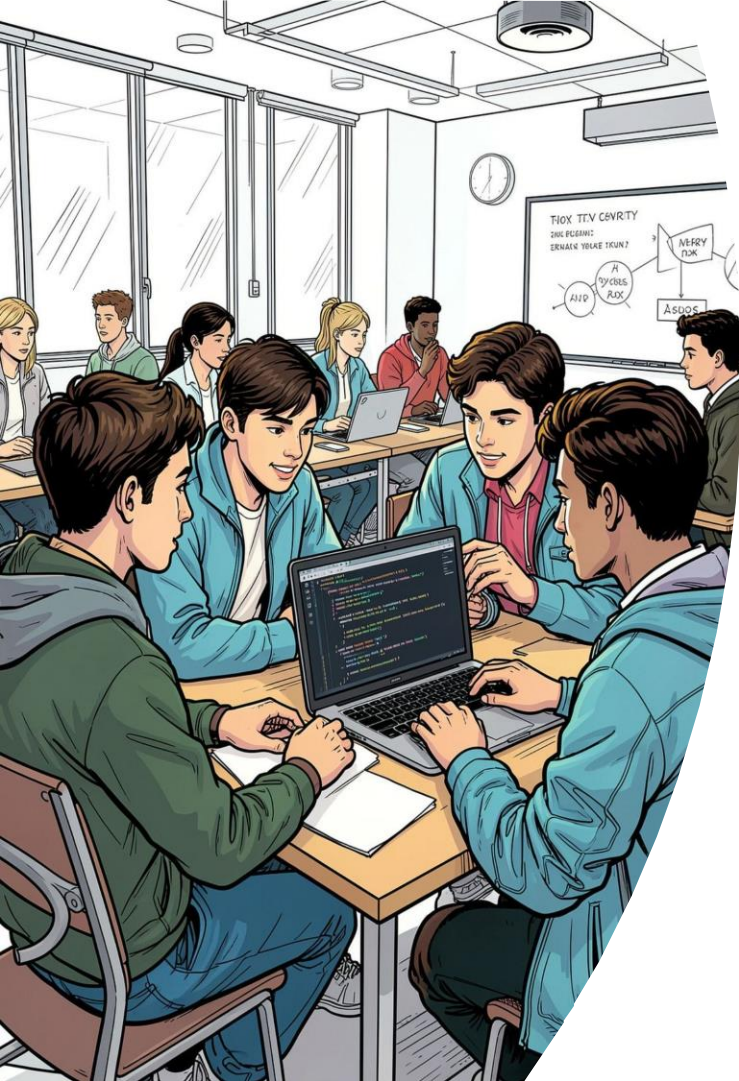
Recommended Approach

The C4 Model and practical examples for capstone projects

05

Tools & Best Practices

Free tools, documentation tips, and defense strategies



Capstone Project & Research

2: Context

Capstone Project and Research 2 is the culminating subject that integrates **analysis, design, development, and research** into a single coherent project.

Typical Deliverables

Problem statement, requirements, system design, implementation, testing, documentation, and oral defense

Where Architecture Fits

System architecture is a core part of the **System Design chapter** (often Chapter 3), showing how components interact to satisfy functional and non-functional requirements

Why System Architecture Matters

A well-crafted architecture diagram is more than a requirement — it demonstrates **engineering maturity** and guides every phase of your project.



Communicates Structure

Conveys high-level design to advisers, panels, and team members at a glance



Guides Technology Choices

Shapes decisions around scalability, security, and maintainability from the start



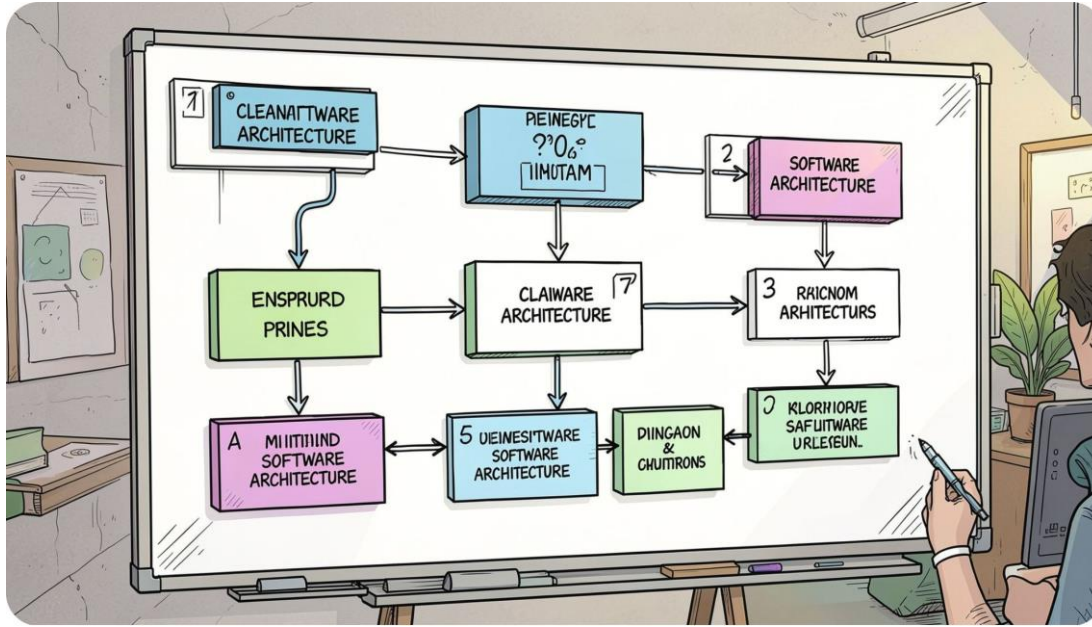
Reduces Rework

Clear architecture prevents costly implementation mistakes and redesign cycles



Required in Rubrics

Design quality and documentation completeness are scored in most capstone evaluations



DEFINITION

What Is a System Architecture Diagram?

A **visual representation** of your system's structure — its components, relationships, and interactions. It focuses on **major building blocks**, not low-level code details.

Diagrams can be free-form or standardized (UML deployment, C4, layered boxes).

→ What are the main parts?

→ How do they communicate?

→ Where does data live?

→ What external systems are involved?

Common Types Relevant to Capstone

Choose the diagram type that best matches your project's complexity and your panel's expectations.



Context Diagram

System as a black box with external actors and systems surrounding it



Container / Component

Major deployable units: web app, API, database, mobile app



Layered Architecture

Presentation → Business Logic → Data Access → Database



Client-Server / N-Tier

Browser or mobile client, application server, database server



Microservices

Independent services with API Gateway — for advanced projects

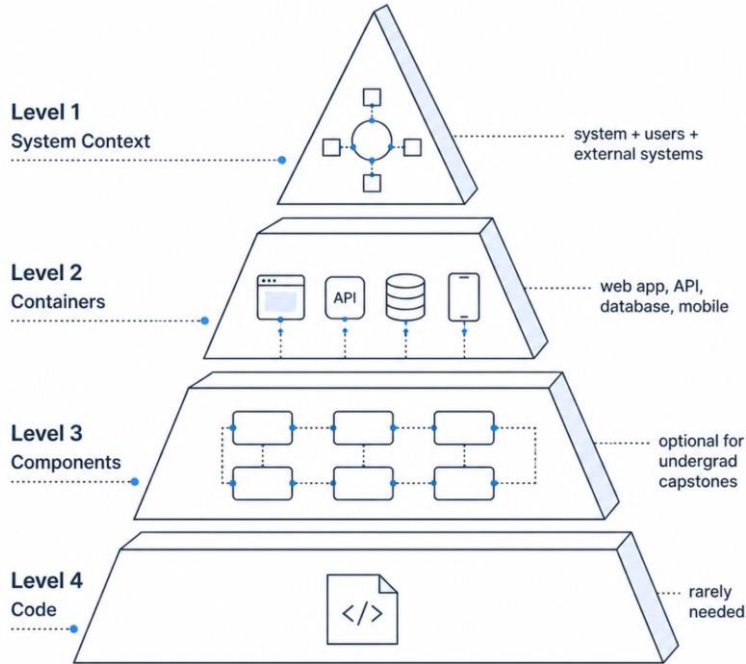


Deployment Diagram

Physical servers, cloud infrastructure, and devices

Recommended Approach: The C4 Model

The **C4 Model** offers a student-friendly, progressive zoom — from a high-level overview down to technical detail. Panels appreciate the clarity it provides.



Level 1 – System Context

Show your system, its users, and external systems. Ideal for panel overviews.

Level 2 – Containers

Web app, API, database, mobile — the main deployable units. Best for technical discussions.

Level 3 & 4 – Components & Code

Optional for most undergrad capstones. Use only if your project demands it.

Architecture Examples for Capstone Projects

Layered Architecture

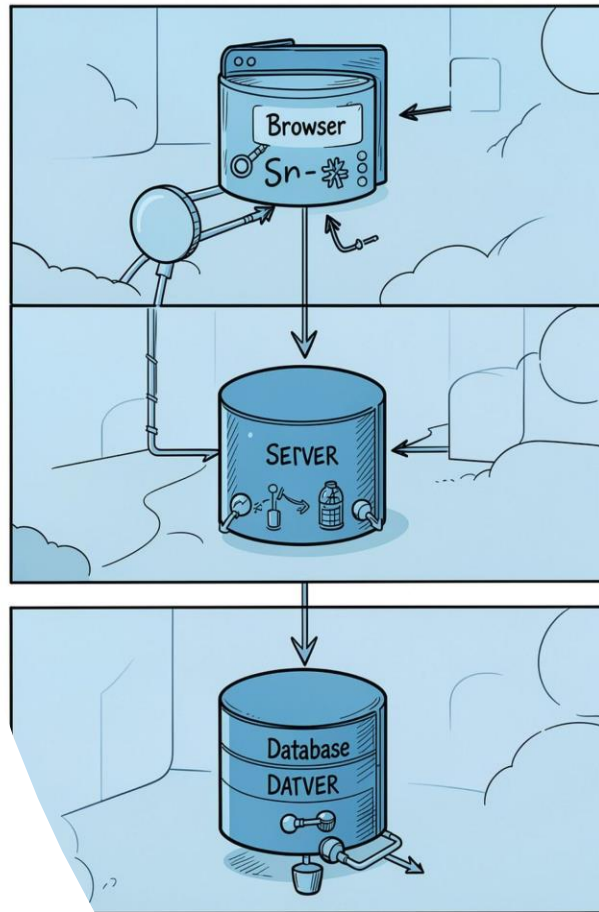
Presentation (React/Mobile) →
Business Logic (REST API) →
Data Access (ORM) → Database
(MySQL/PostgreSQL). Ideal for
SIS, Library, or Inventory
systems.

Client-Server / 3-Tier

Client (Browser/Mobile) → Server
(Node.js/Django/Laravel) →
Database. Add auth, file storage,
or notification services as
needed.

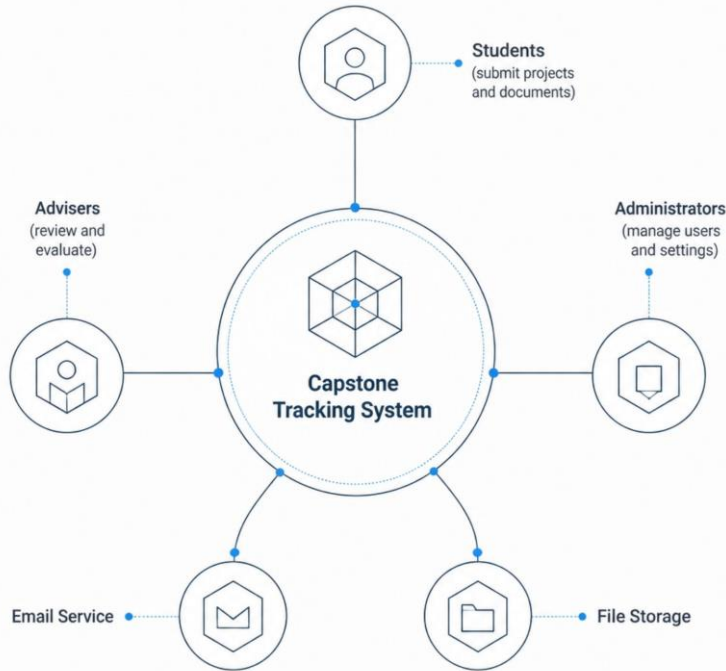
Modern / Cloud-Friendly

Frontend → API Gateway → Microservices
(User, Order, Notification) →
Databases + Message Queue. Suitable for e-commerce, booking, or IoT
dashboards.



Concrete Example: Capstone Tracking System

A typical 3-tier system with JWT authentication — a strong reference model for most capstone projects.



Technology Stack

- **Frontend:** React web app
- **Backend:** Node.js/Express or PHP/Laravel REST API
- **Database:** MySQL (users, projects, submissions, evaluations)
- **External:** Email service, file storage for documents
- **Auth:** JWT token-based authentication

☑ Include both a context diagram and a container diagram in your System Design chapter for full coverage.

Tools, Best Practices & Defense Tips

Recommended Tools

draw.io / diagrams.net

Free, browser-based,
exports PNG/SVG

Lucidchart / Creately

Education plans available,
collaborative

StarUML / PlantUML

UML-focused, great for
formal diagrams

Structurizr / C4- PlantUML

C4-specific tooling for
structured models

Best Practices

- Keep diagrams **clean and readable** — consistent colors, clear labels
- Match detail level to your audience (context for overview, containers for technical discussion)
- Document **technology choices and rationale**
- Update diagrams as the design evolves

Defense Tips

- Place the main diagram **early** in the System Design chapter
- Walk the panel through data flow step by step
- Be ready to justify your architecture over alternatives
- Prepare a **simplified version** for presentation slides

👉 **Key Takeaway:** Start simple (context + containers), use standard readable styles, and always support your diagram with clear written explanations.